

ARCHITECTURE RECOMMENDATION

account\_inquiry

In-app assistant answering questions about account balances, cost basis, contributions and balance changes from deterministic books-and-records APIs, with FAQ explanations.

|                            |                                          |
|----------------------------|------------------------------------------|
| Recommended architecture   | Single agent with tools (single)         |
| Budget verdict             | Exceeds the budget                       |
| Estimated p50 latency      | ~7 s of a 5 s budget                     |
| Estimated cost per request | \$0.003 at list prices, no caching       |
| Tokens vs. one model call  | 1x (2 model calls)                       |
| Score                      | +2.1 (fit +3.5, latency -0.9, cost -0.5) |
| Currently implemented      | Single agent with tools (matches)        |

Recommended: **Single agent with tools** (score +2.1; est. p50 latency ~7s vs budget 5s, ~1x tokens of one call). It exceeds the latency budget and no viable topology fits it: relax the budget, cut tool calls/reading per request, use a smaller model, or move work off the request path. This matches the topology the repository already implements; focus on the cross-cutting recommendations.

1. Why this architecture

- **+2.0** Simple task with a few tools: a single agent loop is enough. [OpenAI, A practical guide to building agents]
- **+2.0** Unpredictable path: the model must choose steps at run time, which is the definition of an agent. [Anthropic, Building effective agents]
- **+2.0** Multi-hop retrieval: the model must decide follow-up queries from earlier results (agentic retrieval). [Anthropic, Effective context engineering for AI agents]
- **+2.0** A 88% single-agent baseline is the regime where one agent plus a better model/effort/tools wins. [Google/MIT, Towards a Science of Scaling Agent Systems]

Counter-signals

- **-1.5** Serial round trips do not fit an interactive budget. [Anthropic, Building effective agents]
- **-1.5** Overlapping tools are OpenAI's split trigger (systems fail with <10 overlapping tools yet manage 15+ distinct ones): first consolidate/rename tools; if that fails, give each specialist a disjoint tool set. [OpenAI, A practical guide to building agents]
- **-1.5** Agentic retrieval loops run 3-20x the latency of single-pass RAG; an interactive budget forces single-pass retrieval with escalation only on failure signals (adaptive RAG). [Agentic vs classic RAG guidance (see docs/industry\_guidance.html)]

2. Repository scan

7 files scanned under /Users/saxena/Desktop/ClaudeCode\_GIT/AgentTopologyBuilder/case\_study/03\_account\_inquiry. Frameworks: anthropic-sdk. Tools found: 7. Knowledge sources: pgvector, sql. Side effects: none. Current topology as implemented: **single\_agent** (A tool-use loop (or tool definitions) with no delegation.).

Inferred profile fields

| Field               | Value      | Conf. | Reason                                                                         |
|---------------------|------------|-------|--------------------------------------------------------------------------------|
| tool_count          | 7          | 80%   | 7 tool definitions found (decorators, schemas, MCP tools, OpenAPI operations). |
| tool_calls_per_task | 4          | 30%   | Rough: 7 tools and 0 tool-handling call sites; verify against traces.          |
| tool_dependency     | sequential | 40%   | No concurrency primitives around tool calls.                                   |

| Field                   | Value       | Conf. | Reason                                                                                                      |
|-------------------------|-------------|-------|-------------------------------------------------------------------------------------------------------------|
| tool_side_effects       | read_only   | 60%   | No write/side-effect integrations detected.                                                                 |
| knowledge_sources       | 2           | 60%   | Retrieval integrations: pgvector, sql                                                                       |
| retrieval_depth         | multi_hop   | 50%   | Derived from sources (2) and whether a loop can issue follow-up queries.                                    |
| context_tokens_per_task | 24500       | 30%   | Rough function of sources and tools; replace with measured input tokens per request.                        |
| scope_breadth           | 1           | 40%   | 1 file(s) define system prompts/instructions; routing/handoffs raise this.                                  |
| parallel_subtasks       | 1           | 40%   | Single tool loop.                                                                                           |
| steps_predictable       | False       | 60%   | The model chooses steps at run time.                                                                        |
| task_complexity         | moderate    | 40%   | Tool loop with 7 tools.                                                                                     |
| latency_budget_s        | 10.0        | 40%   | Request/response serving implies an interactive budget.                                                     |
| interaction             | single_turn | 40%   | Serving surface and memory usage.                                                                           |
| verifiability           | partial     | 50%   | Tests or schema validation present.                                                                         |
| tool_overlap            | True        | 50%   | Similarly named tools: get_balances, get_contributions_ytd, get_cost_basis, get_positions, get_transactions |

### Evidence (first hits per category)

| Category       | Label                 | Location                      | Snippet                                                                                  |
|----------------|-----------------------|-------------------------------|------------------------------------------------------------------------------------------|
| frameworks     | anthropic-sdk         | app/assistant.py:2            | import anthropic                                                                         |
| frameworks     | anthropic-sdk         | app/assistant.py:10           | client = anthropic.Anthropic()                                                           |
| agent_patterns | tool runner           | app/assistant.py:24           | runner = client.beta.messages.tool_runner(                                               |
| agent_patterns | tool runner           | app/assistant.py:28           | final = runner.until_done()                                                              |
| serving        | http-api              | app/assistant.py:9            | app = FastAPI()                                                                          |
| serving        | websocket/streaming   | app/assistant.py:1            | """Account assistant: one tool-use loop behind a websocket, with Redis-backed memory.""" |
| frameworks     | anthropic-sdk         | app/tools.py:3                | from anthropic import beta_tool                                                          |
| tools          | python decorator tool | app/tools.py:7                | get_balances                                                                             |
| tools          | python decorator tool | app/tools.py:13               | get_positions                                                                            |
| tools          | python decorator tool | app/tools.py:19               | get_cost_basis                                                                           |
| tools          | python decorator tool | app/tools.py:25               | get_transactions                                                                         |
| tools          | python decorator tool | app/tools.py:31               | get_contributions_ytd                                                                    |
| retrieval      | pgvector              | app/tools.py:4                | from pgvector.psycopg import register_vector                                             |
| retrieval      | pgvector              | app/tools.py:45               | """Semantic search over the approved FAQ / explanations knowledge base (pgvector)."""    |
| retrieval      | sql                   | app/tools.py:4                | from pgvector.psycopg import register_vector                                             |
| quality        | tests                 | tests/test_numbers_match.py:1 | def test_answers_only_use_tool_numbers():                                                |

## 3. Options compared

Score = structural fit + latency adjustment + cost adjustment. Latency is the estimated p50 on the critical path; cost sums every call at list prices. Non-viable options cannot do the job and are listed last.

| # | Architecture            | Score | Fit  | Latency | Budget  | Cost/req | Tokens | Calls |
|---|-------------------------|-------|------|---------|---------|----------|--------|-------|
| 1 | Single agent with tools | +2.1  | +3.5 | ~7 s    | exceeds | \$0.003  | 1x     | 2     |

| #  | Architecture                                                                                                                            | Score | Fit   | Latency | Budget   | Cost/req | Tokens | Calls |
|----|-----------------------------------------------------------------------------------------------------------------------------------------|-------|-------|---------|----------|----------|--------|-------|
| 2  | Router / classifier dispatch                                                                                                            | +1.2  | +3.0  | ~7 s    | exceeds  | \$0.004  | 2x     | 3     |
| 3  | Parallel voting / ensemble                                                                                                              | -2.9  | +1.0  | ~8 s    | exceeds  | \$0.017  | 7x     | 10    |
| 4  | Evaluator-optimizer loop                                                                                                                | -6.1  | -1.5  | ~31 s   | far over | \$0.011  | 4x     | 9     |
| 5  | Prompt chain (sequential workflow)                                                                                                      | -6.7  | -3.5  | ~18 s   | far over | \$0.005  | 2x     | 3     |
| 6  | Orchestrator + worker subagents                                                                                                         | -16.9 | -12.5 | ~30 s   | far over | \$0.009  | 3x     | 6     |
| 7  | Single model call<br><small>not viable: needs tool calls; a single call cannot act</small>                                              | -1.0  | -2.0  | ~2 s    | fits     | \$0.002  | 1x     | 1     |
| 8  | Parallel sectioning (code-defined fan-out)<br><small>not viable: nothing to fan out</small>                                             | -3.3  | +0.0  | ~11 s   | far over | \$0.007  | 3x     | 5     |
| 9  | Specialist handoffs (decentralised)<br><small>not viable: no second specialist to hand off to</small>                                   | -10.3 | -7.0  | ~12 s   | far over | \$0.006  | 3x     | 5     |
| 10 | Hierarchical teams (orchestrator -> leads -> workers)<br><small>not viable: no domains or sub-task volume to justify two levels</small> | -25.7 | -20.0 | ~47 s   | far over | \$0.020  | 7x     | 14    |

Latency/accuracy frontier (no option is both faster and a better structural fit): Single agent with tools (~7 s, fit +5.0). Fastest viable within budget: **none**. Strongest structural fit: **Single agent with tools**.

Runners-up

- **Router / classifier dispatch** (score +1.2, ~7 s, 2x tokens): Routing to specialists gives each a small, non-overlapping tool set; Routing easy requests to a small model is the cheapest accuracy-preserving lever.
- **Parallel voting / ensemble** (score -2.9, ~8 s, 7x tokens): High-stakes bounded decision: several independent votes/judges raise confidence at parallel (not serial) latency cost.
- **Evaluator-optimizer loop** (score -6.1, ~31 s, 4x tokens): High accuracy priority and an objective check: generate -> evaluate -> refine converts test/rubric feedback into measurable gains.

4. Decomposition plan: Single agent with tools

One model in a tool-use loop: it decides which tools to call, reads results, and iterates until done.

opus

sonnet

haiku

code

Agent

acme-onprem-70b · effort n/a

| Component | Role                                                    | Model           | Effort | Tools                                                                                                                                       | Parallel |
|-----------|---------------------------------------------------------|-----------------|--------|---------------------------------------------------------------------------------------------------------------------------------------------|----------|
| Agent     | Plans and executes the whole task in one tool-use loop. | acme-onprem-70b | n/a    | explain_balance_chan<br>ge, faq_lookup,<br>get_balances,<br>get_contributions_ytd,<br>get_cost_basis,<br>get_positions,<br>get_transactions | -        |

## 5. Model selection per role

Catalog case-study-ecosystem; data class **restricted**; verified entries only. Providers used: self\_hosted.  
Estimated per request on the chosen models: ~7 s, \$0.003.

| Role                  | Needs                          | Chosen          | Effort | Fallback | Est./call         | Alternatives |
|-----------------------|--------------------------------|-----------------|--------|----------|-------------------|--------------|
| agent<br>single_agent | tier ≥3; 5s; 10,050 ctx; tools | acme-onprem-70b | n/a    | -        | ~5.0s<br>\$0.0053 | -            |

- **agent:** Tier 4 vs required 3; est. 5.0s of a 5.0s share; \$0.0053 per call. measured conversation accuracy 90%

## 6. Cross-cutting recommendations

- **Build a ~20-query eval before adding any structure.** Every source agrees: measure a single agent/call first, then add topology only where the eval shows it failing. Above a ~45% single-agent baseline, adding agents tended to hurt in the scaling study. Keep the eval to compare latency and accuracy across topologies. [Google/MIT, Towards a Science of Scaling Agent Systems]
- **Explicit termination limits on every loop.** Max turns/tool calls per agent, max evaluator iterations, max handoffs, max concurrent subagents. 'Unaware of termination' and 'step repetition' account for ~28% of multi-agent failures in MAST. [Cemri et al., Why do multi-agent LLM systems fail? (MAST)]
- **Stream responses.** Perceived latency drops to time-to-first-token; long outputs must stream to avoid timeouts. [Anthropic, Effective context engineering for AI agents]
- **Prompt caching on the stable prefix (system prompt, tool schemas, reference docs).** Multi-turn and multi-call topologies re-send the same prefix on every call; caching cuts cost up to ~90% and time-to-first-token. [Anthropic, Effective context engineering for AI agents]
- **Parallel tool use inside each agent turn.** Independent tool calls issued in one assistant turn cut rounds by 2-4x; return all results in one message. [Anthropic, Effective context engineering for AI agents]
- **Context editing (clear stale tool results/thinking).** Keeps long loops lean without summarisation losses. [Anthropic, Effective context engineering for AI agents]
- **Memory across sessions (file/DB-backed).** State that must outlive one conversation should be written to memory, not carried in context. [Anthropic, Effective context engineering for AI agents]
- **Agentic retrieval capped at ~4 iterations: start broad, then narrow.** Multi-hop retrieval buys accuracy (27% → 43% EM on HotpotQA) at 3-20x latency; cap the loop and prefer short broad queries first, narrowing progressively. [Agentic vs classic RAG guidance (see docs/industry\_guidance.html)]
- **Adaptive RAG: single-pass by default, escalate on failure signals.** Route simple queries to one retrieval; escalate to the multi-hop loop only on missing citations, low retrieval confidence, contradictions or repeated follow-ups. [Agentic vs classic RAG guidance (see docs/industry\_guidance.html)]
- **Consider a difficulty router in front.** Send easy requests to a small model and hard ones to the full path; the largest cost lever after caching. [Anthropic, Building effective agents]

## 7. Estimate assumptions

- 1 sequential tool round(s) for 2 tool calls (batching by dependency=independent).
- Serving assumptions (reference model per capability level from the catalog): acme-onprem-70b ~0.5s TTFT, ~90 tok/s; acme-onprem-70b ~0.5s TTFT, ~90 tok/s; acme-onprem-70b ~0.5s TTFT, ~90 tok/s; tool call ~0.3s.
- Reading load per task 6,000 tokens (context pressure: low); output type short\_answer.
- Latency is the critical path (parallel branches count once); cost sums every call at list prices without caching. Treat both as order-of-magnitude.

## Appendix A. Workload profile used

| Field                   | Value               | Field               | Value                                                                                                                                    |
|-------------------------|---------------------|---------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| name                    | account_inquiry     | task_complexity     | simple                                                                                                                                   |
| steps_predictable       | False               | scope_breadth       | 1                                                                                                                                        |
| parallel_subtasks       | 1                   | output_type         | short_answer                                                                                                                             |
| tool_count              | 7                   | tool_calls_per_task | 2                                                                                                                                        |
| tool_dependency         | independent         | tool_side_effects   | read_only                                                                                                                                |
| tool_overlap            | True                | tool_latency_s      | 0.3                                                                                                                                      |
| knowledge_sources       | 2                   | retrieval_depth     | multi_hop                                                                                                                                |
| context_tokens_per_task | 6000                | accuracy_priority   | 5                                                                                                                                        |
| single_agent_baseline   | 0.88                | verifiability       | strong                                                                                                                                   |
| error_recoverability    | easy                | latency_budget_s    | 5.0                                                                                                                                      |
| interaction             | multi_turn          | requests_per_day    | 200000                                                                                                                                   |
| cost_sensitivity        | 5                   | human_in_loop       | False                                                                                                                                    |
| data_sensitivity        | restricted          | tools               | ['explain_balance_change', 'faq_lookup', 'get_balances', 'get_contributions_ytd', 'get_cost_basis', 'get_positions', 'get_transactions'] |
| sources                 | ['pgvector', 'sql'] |                     |                                                                                                                                          |

## Appendix B. Sources

- **ANTHROPIC\_BEA**: Anthropic, Building effective agents
- **ANTHROPIC\_CTX**: Anthropic, Effective context engineering for AI agents
- **ANTHROPIC\_MA**: Anthropic, Managed Agents multiagent guidance
- **ANTHROPIC\_MAR**: Anthropic, How we built our multi-agent research system
- **COGNITION**: Cognition, Don't build multi-agents
- **MAST**: Cemri et al., Why do multi-agent LLM systems fail? (MAST)
- **OPENAI\_PG**: OpenAI, A practical guide to building agents
- **RAG**: Agentic vs classic RAG guidance (see docs/industry\_guidance.html)
- **SCALING**: Google/MIT, Towards a Science of Scaling Agent Systems

Full evidence and the consolidated decision framework: docs/industry\_guidance.html and docs/METHODOLOGY.html in the VG Select: 3A repository.